SlideSolve sliding tile puzzle solver

Derek Song

Advanced Algorithms Programming Assignment 1

Track B – A* Search

1 What I built

SlideSolve is my Python command-line sliding-tile puzzle solver, developed with substantial assistance from OpenAI Codex. It uses A* to find a shortest sequence of legal moves, prints the solution, and reports search statistics. It can also generate reproducible solvable puzzles. The implementation supports the 8-puzzle on a 3 by 3 board and the 15-puzzle on a 4 by 4 board. The performance evaluation concentrates on the 8-puzzle; support for 4 by 4 input does not imply that arbitrary difficult 15-puzzles can be solved within practical memory limits.

I selected A* puzzle solving and guided the development through detailed prompts, suggestions, and review. I supplied the assignment requirements and accurate examples of the behaviour I was seeing, asked for explanations of specific code, and raised questions about whether the implementation met the intended scope. I also performed my own manual output testing with different board sizes and tile combinations, looked for possible issues, and brought unexpected results back for investigation.

The project targets Python 3.11 or newer and has no third-party runtime dependencies. The main search is implemented directly with tuples, dictionaries, and Python's heapq priority queue rather than calling an existing search library. Codex proposed and implemented these design choices; my role included reviewing the resulting behaviour, testing the output, questioning the implementation, and suggesting revisions.

Representation and organisation

A board is an immutable flat tuple of integers, with zero representing the blank. For example, `(1, 2, 3, 4, 0, 6, 7, 5, 8)` represents the two-move demonstration puzzle. Tuples can be dictionary keys and prevent accidental mutation of stored states. The width is inferred from the tile count or supplied explicitly. Moves name the blank's direction, not the numbered tile's direction.

The source is divided by responsibility. `board.py` handles parsing, validation, neighbouring states, and formatting. `heuristics.py` contains the three estimates. `solvability.py` checks inversion parity. The core algorithm and path reconstruction are in `astar.py`. `generator.py` creates puzzles, `cli.py` handles the interface, and `benchmark.py` performs the additional evaluation.

Search and correctness

A* orders candidates by f = g + h. Here g is the route length so far and h estimates the remaining number of moves. Every legal slide costs one. The `frontier` heap stores entries containing the priority, an insertion counter, the route cost, and the board. The counter gives deterministic ordering when priorities tie. `g_score` records the cheapest discovered cost for each board, while `came_from` records parent boards and moves for reconstruction.

The best-known-cost invariant is that each `g_score` value is the cheapest route cost discovered so far for that board. The starting board receives zero. When a neighbour is considered, the solver accepts its proposed cost only if it is strictly cheaper than the recorded value. An accepted improvement updates the cost and parent and pushes a new heap entry.

An old entry is not edited in place. When removed from the heap, its stored cost is compared with the current `g_score`. If they differ, the entry is stale and is skipped. The check happens before the goal check and neighbour expansion. Once a current goal entry is removed, the solver follows parent links backwards and reverses the resulting sequence.

Three heuristics are available. Zero makes A* behave as uniform-cost search. Misplaced tiles counts numbered tiles outside their goal positions. Manhattan distance, the default, sums their horizontal and vertical distances to the goal. The latter two exclude the blank. They are admissible and consistent for unit-cost sliding puzzles: each move affects one numbered tile by one square, so these estimates do not overstate the required moves and satisfy the consistency inequality across a legal edge. With the implemented handling of improved routes, the registered heuristics preserve A*'s shortest-path guarantee. A user-supplied heuristic is responsible for meeting the necessary conditions; an arbitrary callable is not guaranteed to return optimal results.

Validation and practical limits

Input must form a square board of width at least two and contain every integer from zero through the tile count minus one exactly once. Odd-width puzzles are solvable when their inversion count is even. For even widths, the inversion count plus the blank's row from the bottom must be odd for the standard goal. These checks reject impossible input before search.

A* can consume substantial memory because it retains discovered states and may keep stale heap entries until they are removed. The expansion-limit option bounds the number of states expanded, not memory use or elapsed time. The implementation is a teaching and demonstration tool, not a specialised solver for the hardest 15-puzzles.

2 The working tool

Interface decisions

The command-line interface accepts comma-separated or whitespace-separated tiles. It offers a heuristic selection, full intermediate boards, compact output, and an expansion limit. These options expose the algorithm's behaviour without requiring a graphical interface. Output includes the move sequence, solution length, expanded states, generated transitions, maximum frontier size, and runtime. Expanded states exclude the goal; generated transitions count considered neighbour edges, including duplicates. Maximum frontier size counts heap entries, including stale ones, and is not a measurement of bytes used.

Run from the repository folder after making the source package visible:

```
$env:PYTHONPATH = "src"
python -m slidesolve solve "1,2,3,4,0,6,7,5,8" --show-boards
```

The interface reports malformed or unsolvable input with exit status 2 and an expansion-limit stop with status 3. A successful solve returns status 0. The generator makes legal moves from the goal, optionally with a seed. This guarantees solvability, but its scramble length is not necessarily the shortest solution length.

Worked example

The demonstration starts with this board, where an underscore represents the blank:

```
1 2 3
4 _ 6
7 5 8
```

Tile 5 is one row from its goal and tile 8 is one column from its goal, so Manhattan distance is two.

The initial priority is two because the route cost is zero. Moving the blank down produces `1,2,3,4,5,6,7,0,8`, with route cost one and estimate one. Moving it right reaches the goal. The output is `down right`, with solution length two, two expanded states, and seven generated transitions. All three boards are printed with `--show-boards`.

For a 15-puzzle example, the input `1,2,3,4,5,6,7,8,9,10,11,12,13,0,14,15` produces `right right`. This uses the same search implementation and heuristics; it does not require a separate 4 by 4 solver.

Testing and additional evaluation

Testing included both my own manual checks and the automated suite generated and run by Codex. I ran the program with different board sizes and tile combinations and inspected its output. I searched for possible issues, questioned unexpected results, and provided the relevant input and error messages for investigation. I also reviewed the generated tests and judged that they covered appropriate cases, including invalid input, optimal solution lengths, and legal moves.

One example was my 4 by 4 input `1,2,3,0,5,6,7,8,9,10,11,12,13,14,4,15`. The program reported that it could not reach the standard goal, so I raised that result for explanation. The board is unsolvable under the standard rules; its rejection was not evidence that the solver lacked 15-puzzle support. I also questioned the original documentation's focus on the 8-puzzle when the topic description mentioned the 15-puzzle. This prompted clarification of the supported sizes and their practical limits.

The recorded walkthrough describes the 46 tests present at recording time. Four subsequent 15-puzzle tests increased the suite to 50, and another Codex agent then added six further tests, bringing the final total to 56. All 56 passed. The program source code has not changed as part of these additional tests. My manual checks complemented the automated suite but were not an exhaustive correctness proof or the source of the benchmark measurements.

The original checks include exhaustive distances for all 12 reachable 2 by 2 boards, heuristic admissibility and consistency on that small state space, seeded 3 by 3 cases, and a known 31-move 8-puzzle. The breadth-first search used for the original small-board oracle is a separate search algorithm but shares the production neighbour function. The final 4 by 4 tests address that limitation by independently generating legal swaps and checking all three heuristics against exact distances through depth five. This is shallow coverage, not exhaustive coverage of the 15-puzzle.

The code was also tested by another Codex agent in a separate conversation from the development conversation. Its initial verification found that all 50 existing tests passed, 3 by 3 solving returned the correct optimal two-move solution, and 4 by 4 solving returned the correct solution. Seeded puzzle generation, CLI help, and board display also worked.
That agent subsequently added test_additional_confidence.py with six tests. An independent breadth-first search, using its own move generator, enumerates all 181,440 reachable 3 by 3 states. The tests check solvability for every reachable state and its unsolvable twin, and check misplaced-tile and Manhattan admissibility and consistency across the entire reachable state space. Manhattan A* is compared with an exact-distance representative at every depth from 0 to 31; all three heuristics are compared at depths 0, 5, 10, 15, 20, 25, 30, and 31. Another test checks 100 seeded 4 by 4 generations by independently replaying their moves. The agent reported that all 56 tests passed in 9.145 seconds. It added test code only; no program source code was changed. These checks do not mean A* was run on every reachable 3 by 3 board.

That agent could not access the Windows `python` alias in its environment, so it used the installed Python 3.11 interpreter through `py -3.11`. This was an environment issue rather than a program

defect. The separate pass provided additional evidence for the tested behaviour; it did not establish correctness for every possible input.

Codex also built an optional heuristic evaluation. Breadth-first search establishes exact 8-puzzle distances before sampling three boards at each depth of 5, 10, 15, 20, 25, and 30, using seed 20260927. Each heuristic runs three times on each board: 18 distinct puzzles and 162 runs in total. Every returned solution length matches the breadth-first-search distance.

Exact depth	Zero mean expanded	Misplaced mean expanded	Manhattan mean expanded
5	46.0	7.0	5.0
10	597.3	39.3	19.3
15	7553.7	372.3	61.7
20	47115.3	3409.0	606.7
Exact depth	Zero mean expanded	Misplaced mean expanded	Manhattan mean expanded
25	152421.3	28515.0	2164.3
30	181376.7	122017.3	21285.0

At depth 30, Manhattan distance reduces average expansions by 88.3 percent compared with zero, while misplaced tiles reduces them by 32.7 percent. These measurements illustrate why the more informative estimate can reduce search effort.

The sample is small. Repeating deterministic searches supplies timing measurements, not additional independent puzzles. The reported median runtime pools nine runs per depth and heuristic; fixed heuristic execution order and background activity may affect timing. The original runtime environment was not fully recorded, so exact timings should not be treated as portable results. Expanded states are the primary metric. These results concern this implementation and sampled 8-puzzles, not general 15-puzzle performance. Raw CSV data, summaries, analysis, and a static workbook are included under `results`.

3 What I learned

Running my own inputs helped me distinguish unexpected output from an implementation defect. When my 4 by 4 example was rejected, I initially needed an explanation of why it could not reach the goal. Investigating that result connected the input checks to the mathematical restrictions of the puzzle. It also showed why supporting a board size and being able to solve every arrangement entered for that size are different claims.

The distinction between a board and an entry in the queue took me time to understand. I asked why finding a better route to an already queued board was tricky and whether popped entries stayed in the heap forever. Popping means removing an entry. The old entry remains only until it is popped, or until the search finishes and the local heap is discarded. More than one entry can describe the same board, but `g_score` records only its best discovered cost.

The eight-moves versus six-moves example helped me connect the code to its purpose. If both routes reach the same arrangement, the possible next moves are identical. There is no advantage to arriving with two extra moves. A new cheaper entry can coexist with an old one, and the stale-entry check prevents the old cost from being used for another expansion.

I also asked several questions about excluding the blank from Manhattan distance. The key example is a solved board except for the last row `7, blank, 8`. One move puts both the blank and tile 8 in their goal positions. Counting both distances would give two even though only one move is needed. This made overestimation more concrete than simply memorising the word admissible. Separately, the expression `tile - 1` assumes a numbered tile; applying it to zero gives negative one and the wrong goal-position calculation.

Providing detailed prompts was part of the development process. I supplied the walkthrough requirements, requested explanations of specific lines and concepts, and suggested revisions to the demonstration and report. Giving the actual board input or error message made a question more precise and helped focus the investigation. Preparing the recording then required me to explain the algorithm while pointing to its code: where the starting cost is set, where cheaper routes replace earlier costs, and where stale entries are rejected.

4 AI use

I used OpenAI Codex and ChatGPT substantially throughout the project. They helped explore topic options and scope, proposed the architecture, generated the implementation and automated tests, ran checks, created the CLI and benchmark, produced result files, and drafted documentation and the report. They also helped draft a guide for the walkthrough video, as well as explain unfamiliar concepts. A separate Codex agent tested the code in another conversation, added six additional confidence tests, and reported that all 56 tests passed without changes to the program source code.

My contributions included choosing A* puzzle solving, guiding the work with detailed and accurate prompts, making suggestions, reviewing the generated tests, and conducting my own manual output testing with different board sizes and tile combinations. I searched for and identified possible issues to investigate, supplied concrete inputs and error messages, and asked follow-up questions rather than accepting unexpected behaviour without explanation. I also checked the project's scope against the assignment, requested clearer explanations and revisions, ran the demonstration, and recorded the walkthrough in my own voice.

Three clear problems occurred in AI-generated work. First, the original validator rejected an explicitly supplied width of one but accepted a one-tile board when the width was inferred. Codex found this during testing (which I had prompted for), added the missing rejection, and retained a regression test. I did not independently discover or implement that fix. It is an example of why tests need to cover different routes through validation.

Second, the initial formatter used a Unicode middle dot for the blank. Although its string test passed, I noticed the captured terminal output showed a replacement character. Codex then changed the marker to an ASCII underscore and updated the test. The precise source of the encoding or display problem was not isolated, so I do not attribute it to a proven terminal setting. The practical response was to retain the simpler marker, which is visible in my demonstration.

A third issue was that the AI-generated draft report understated my contributions. It omitted my manual testing with different board sizes and tile combinations, investigation of possible issues, suggestions, and detailed prompts. I identified these omissions and requested revisions so the report accurately distinguished my work from the work performed by Codex.

The division of work matters: I performed manual testing and review, raised possible issues, and directed revisions, while Codex generated and executed the automated suite and implemented the fixes described above. Reviewing those tests did not mean that I wrote them myself. Equally, using AI to generate the code did not replace my own testing, investigation, suggestions, or responsibility for understanding the submitted work.

My strongest explanation is of the central search flow: the meanings of f, g, and h; the queue and cost dictionary; stale entries; path reconstruction; and excluding the blank. These are the topics I worked through in the conversation and walkthrough, as well as my own research. I relied more heavily on AI for the precise even-width parity rule, the benchmark implementation, spreadsheet generation, and document formatting. Tests provide evidence about those parts, but they are not a substitute for my own derivation or familiarity.
The report and recording should be read with that division of work in mind.